Rambus

Security IP
EIP-130

Root of Trust engine
Driver Development Kit v3.1

Test Specification

Document Revision: A
Document Date: 2022-02-09
Document Number: 007-130310-310
Document Status: Accepted

© Rambus Inc. • rambus.com CONFIDENTIAL

Copyright 2015-2022 Rambus Inc. This document contains information which is proprietary and confidential, and which is protected under patents, copyrights, and/or other IP rights of Rambus Inc. If you are not the intended recipient of this material, please destroy this document and inform Rambus at +1 408 463 8000 or sipsupport@rambus.com immediately.

Rambus Inc. Corporate Headquarters
4453 North First Street, Suite 100
San Jose, CA 95134
Phone: +1 408-462-8000
Website : https://www.rambus.com/
Contact : sipsupport@rambus.com

Rambus ROTW Holding B.V.
Boxtelseweg 26A
5261 NE Vught
The Netherlands
Phone: +31-73-6581900

Table of Contents

Table of Contents 3
List of Tables 4
List of Figures 4
Document Revision History 5
1 Introduction 6
1.1 Purpose and Scope 6
1.2 Scope 6
1.3 Related Documents 6
1.4 Target Audience 6
1.5 Acronyms 6
2 Test Tools 8
2.1 VAL Test Tool 8
2.1.1 test_val_k.ko 9
2.1.2 test_val_combined_u 9
2.1.3 test_val_up / test_val_combined_up 10
2.2 Demo Applications 11
2.2.1 DemoApp_VAL 11
2.2.2 DemoApp_VAL_Specials 11
2.2.3 DemoApp_SecureDebug 14
2.2.4 DemoApp_OTPProgram 14
2.2.5 DemoApp_OTPKeyBlob 16
3 Test Types 18
3.1 Known Answer Test 18
3.2 Multiple Application Test 18
4 Test Configuration Parameters 19
4.1 Driver Configuration Parameters 19
4.1.1 Polling or Interrupt 19
4.1.2 Bounce Buffers 19
4.1.3 Performance 19
4.1.4 Endianness Conversion 19
4.1.5 Use TokenID to check DMA ready 19
4.2 Deployment Configurations 19

5 Supported tests and configurations 20
6 Test Suite, Test Cases and Tests 21
6.1 No Operation services 21
6.2 Symmetric Crypto related services 21
6.2.1 Encryption 21
6.2.2 Hash 25
6.2.3 MAC 25
6.2.4 Key Wrap 27
6.2.5 Encrypt Vector for PKI 27
6.3 Asymmetric Crypto related services 28
6.3.1 With Asset use 28
6.3.2 Without Asset use 32
6.4 TRNG (Random) related services 33
6.5 Asset Management related services 33
6.5.1 Asset related 33
6.5.2 Secure Timer 34
6.5.3 OTP 34
6.6 Authentication Unlock/Secure Debug related services 34
6.7 eMMC related services 34
6.8 Special Functions related services 34
6.8.1 Milenage 35
6.9 Service-related services 35
6.10 System related services 35

List of Tables

Table 1 Supported Tests 20
Table 2 Supported Configurations 20

List of Figures

Figure 1 Test Tool kernel mode deployment 9
Figure 2 Test Tool user and kernel mode deployment with UMDevXS 9
Figure 3 Test Tool user and kernel mode deployment with VEX proxy 10

Document Revision History

Doc Rev	Page(s)	Date	Author	Purpose of Revision
A	All	2022-02-09	MHO	• First release for v3.1, based on v3.0 RevC.

1 Introduction

1.1 Purpose and Scope

This document specifies various tests required for validating the functionality and stability of the driver for the *EIP-130 Root of Trust engine* (HW2.x and HW3.x), referred to as the *EIP-130* in this document.

User applications can use the driver's API to leverage the services that are provided by the *EIP-130*. These services allow secure storage of key material and other security sensitive data (Assets) that can be used in various symmetric and asymmetric cryptography, secure debug and secure timer operations that also are provided as services themselves.

1.2 Scope

This document describes the test cases that are used to verify the driver and the helper libraries functionality for *EIP-130* as well as the configurations for these tests that represent typical driver use cases.

1.3 Related Documents

The following documents are part of the *EIP-130* documentation set.

Ref	Document	Document Number
[1]	Security IP Driver Framework Porting Guide	007-900450-304
[2]	EIP-130 DDK User Guide	007-130310-308
[3]	EIP-130 Driver User Guide	007-130310-301
[4]	EIP-130 Firmware Reference Manual	007-130xx0-204[1]
[5]	EIP-130 DDK Regression System README	N.A.

[1] xx refers to the product version's major/minor version digits. So, for example version 3.1 translates to 31

This information is correct at the time of document release. Rambus reserves the right to update the related documents without updating this document. Please contact Rambus for the latest document revisions.

The preferred method for getting technical support is to use our online support system at https://sipsupport.rambus.com. If you do not have an account yet for this system, please contact Rambus technical support (sipsupport@rambus.com).

1.4 Target Audience

This document is intended for the *EIP-130* driver and application developers and users. This document assists the driver developer in testing the *EIP-130* driver.

1.5 Acronyms

Acronym	Description
3DES	Triple-DES
AEAD	Authenticated Encryption with Associated Data
AES	Advanced Encryption Standard
API	Application Program Interface.
ARIA	South-Koran cipher algorithm
CBC	Cipher Block Chaining
CCM	Counter with CBC-MAC
ChaCha20	Stream cipher created by Daniel J. Bernstein
CMAC	Cipher MAC
CTR	CounTeR
DDK	Driver Development Kit

DES	Data Encryption Standard
DMA	Direct memory Access
DRBG	Deterministic Random Bit Generator
ECB	Electronic CodeBook
ECC	Elliptic Curve Cryptography
ECDSA	Elliptic Curve Digital Signature Algorithm
EdDSA	Edwards-curve Digital Signature Algorithm
EIP	Embedded Intellectual Property
FW	Firmware
GCM	Galois/Counter Mode
HMAC	Hash MAC
HUK	Hardware Unique Key (Root Key)
HW	Hardware
IC	Interrupt Coalescing
ICM	Integer Counter Mode
KAT	Known-Answer Test
MAC	Message Authentication Code
Milenage	3GPP authentication and key generation functions
NOP	No OPeration
NRBG	Non-deterministic Random Bit Generator
OAEP	Optimal Asymmetric Encryption Padding
OS	Operating System
OTP	One-Time Programmable
PKA	Public Key Accelerator
PKCS	Public-Key Cryptography Standards
PKI	Public Key Infrastructure
Poly1305	Cryptographic message authentication code created by Daniel J. Bernstein
PSS	Probabilistic Signature Scheme
RSA	Rivest, Shamir, Adleman
TRNG	True Random Number Generator
SIV	Synthetic Initialization Vector
SM2	Chinese asymmetric algorithm
SM3	Chinese symmetric hash algorithm
SM4	Chinese symmetric cipher algorithm
SW	Software
VAL	VaultIP Abstraction Layer (*EIP-130* API)
VEX	VaultIP EXchange
XEX	XOR Encrypt XOR
XOR	eXclusive OR
XTS	XEX Tweakable Block Cipher with Ciphertext Stealing

2 Test Tools

2.1 VAL Test Tool

The VAL Test Tool (`test_val_k.ko`, `test_val_up` and `test_val_combined_u[p]`) is used to verify the VAL API functions that are commonly available and do not change or depend on the Assets located in the OTP. However, there is one exception the VAL Test Tool will probe if the root key (HUK) is available for use. The VAL Test Tool executes one test suite with which the highest possible VAL API coverage is achieved. The test suite consists of multiple test cases that defines the tests to perform. The tests are based on known answer testing. The VAL Test Tool that must be executed depends on driver deployment. The following sub sections provide the possible driver deployments.

The VAL Test Tool executes by default the predefined test suite, but in user mode the execution of the VAL Test Tool can be controlled with the following command line arguments:

-help	Display this information.
-noclean	No test cleanup
-noreset	No *EIP-130* reset during tests is allowed.
-initotp	Invokes an OTP initialization with a default COID and HUK. After a successful initialization of the COID and HUK, the *EIP-130* firmware is reset to re-initialize the Static Asset references.
-trng[=<Samples>]	Invokes a TRNG configuration with default settings. The default number of Samples is 3072.
-case <Test Case>	Define the test case to run.
-test <Test>	Define the test to run.
-iter <Iteration>	Define the iteration of the test/test case to run.
-mode <mode>	Define the test enable mode: all All tests including tests after the specified test single Only the specified test after Including all tests after the specified test
-vector=<Index>	Vector index number [0...].
-repeat	Repeat a vector till error or program is terminated.
-d	Dump test suite only
-v[=<level>]	Set verbose level

2.1.1 test_val_k.ko

The VAL Test Tool executes as a kernel module. This means that the driver and VAL Test Tool both execute in the kernel space.

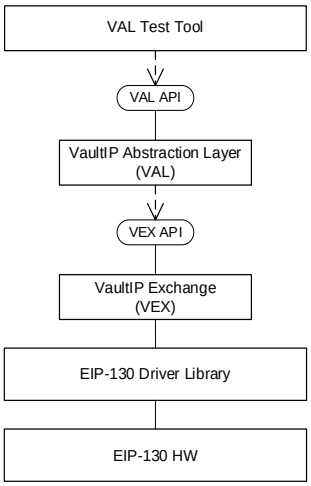


Figure 1 Test Tool kernel mode deployment

2.1.2 test_val_combined_u

This VAL Test Tool combined with the driver executes in the user space. The communication with the *EIP-130* hardware is realized via a general-purpose user space to kernel space proxy (UMDevXS).

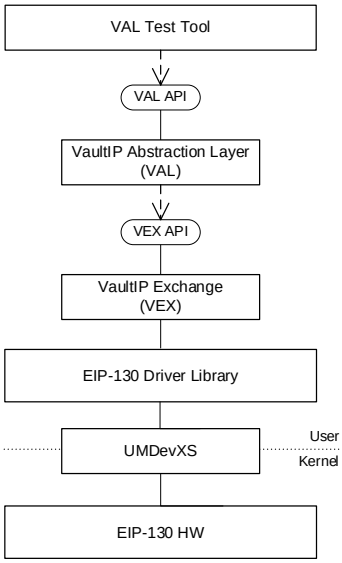


Figure 2 Test Tool user and kernel mode deployment with UMDevXS

2.1.3 test_val_up / test_val_combined_up

This VAL Test Tool executes in the user space and uses a proxy at the driver VEX level to communicate with the driver that executes in the kernel space.

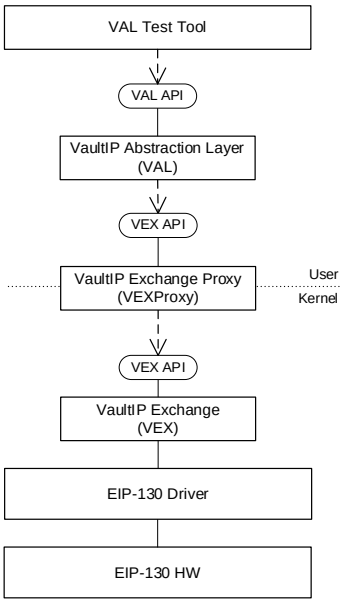


Figure 3 Test Tool user and kernel mode deployment with VEX proxy

2.2 Demo Applications

The DDK provides the following Demo Applications:

DemoApp_VAL	Use example for common VAL API functions
DemoApp_VAL_Specials	Use example for special VAL API functions.
DemoApp_SecureDebug	Use example for Authentication Unlock and Secure Debug.
DemoApp_OTPProgram	Use example for writing an item to OTP.
DemoApp_OTPKeyBlob	Use example for generating OTP key blob with *EIP-130*.

When in this document Demo Application is mentioned, it references to DemoApp_VAL only, because this Demo Application can be used without special requirements regarding the OTP contents or third-party hardware (eMMC). The other Demo Applications can be used but are not part of the standard DDK regression due to their special OTP requirements or OTP zeroization.

2.2.1 DemoApp_VAL

The Demo Application `DemoApp_VAL` (`da_val_k.ko`, `da_val_up` and `da_val_combined_u[p]`) is used to show some typical operations that use the VAL API. Currently, get system version and state, Asset search and a NOP operation are performed to validate the token (mailbox) and data (DMA) interface. The Demo Application executes a predefined of number of operations, but in user mode the initial state of the *EIP-130* can be controlled with the following command line arguments:

-help	Display this information.
-initotp	Initialize the OTP with a default COID and HUK. After successful initialization of the COID and HUK, the *EIP-130* firmware is reset to re-initialize the Static Asset references.
-reset	Reset of the *EIP-130* firmware, so that the Asset Store is cleared. Note that the program stops immediately after the reset is executed.
-trng[=<Samples>]	To invoke a TRNG configuration with default settings. The default number of Samples is 3072.

The Demo Application that must be executed depends on the driver deployment. The following sub sections provide the possible driver deployments.

2.2.1.1 da_val_k.ko

This Demo Application executes as a kernel module. This means that the driver and Demo Application both execute in the kernel space.

Figure 1 shows the deployment, but the Demo Application replaces the VAL Test Tool.

2.2.1.2 da_val_combined_u

This Demo Application combined with the driver executes in the user space. The communication with the *EIP-130* hardware is realized via a general-purpose user space to kernel space proxy (UMDevXS).

Figure 2 shows the deployment, but the Demo Application replaces the VAL Test Tool.

2.2.1.3 da_val_up / da_val_combined_up

This Demo Application executes in the user space and uses a proxy to communicate with the driver that executes in the kernel space.

Figure 3 shows the deployment, but the Demo Application replaces the VAL Test Tool.

2.2.2 DemoApp_VAL_Specials

The Demo Application `DemoApp_VAL_Specials` (`da_val_specials_up` and `da_val_specials_combined_u[p]`) is used to show how to use special VAL API functions that have special OTP requirements

or perform the OTP zeroization. The Demo Application is only built for user mode and the command line arguments control the program behavior. The following command line arguments can be specified:

-help	Display this information.
-asreset	Reset/Clear dynamic Asset Store.
-emmcauthkey	Write eMMC authentication key to OTP. Note that the OTP must be initialized with HUK and COID, the *EIP-130* firmware is reset to re-initialize the Static Asset references.
-emmccheck	Check the eMMC read and write functions based on the eMMC authentication key.
-milenageotp	Write Milenage K and OPc to OTP. Note that the OTP must be initialized with HUK and COID, the *EIP-130* firmware is reset to re-initialize the Static Asset references.
-milenageautn	Check the Milenage AUTN verification sequence based on the Milenage K and OPc.
-milenageauts	Check the Milenage AUTS generation sequence based on the Milenage K and OPc.
-counterotp	Initialize an OTP monotonic counter with AssetNumber 33. Note that the OTP must be initialized with HUK and COID, and the EIP-130 firmware is reset to re-initialize the Static Asset references.
-counterinc[=<Assetnumber>]	Increment the specified monotonic counter. The default AssetNumber is 33.
-initotp	Initialize the OTP with a default COID and HUK. After successful initialization of the COID and HUK, the *EIP-130* firmware is reset to re-initialize the Static Asset references.
-provrndhuk128[=<Samples>]	Initialize the OTP with a random 128-bit HUK based on AES-DRBG without CRC and default COID. There is no OTP key blob generated. Optionally <Samples> specify the TRNG cycle samples for the TRNG Configure/Start (default=3072).
-provrndhuk256[=<Samples>]	Initialize the OTP with a random 256-bit HUK based on AES-DRBG with CRC and default COID. An OTP key blob with HUK is generated. Optionally <Samples> specify the TRNG cycle samples for the TRNG Configure/Start (default=3072).
-ariaprovrndhuk128[=<Samples>]	Initialize the OTP with a random 128-bit HUK based on ARIA-DRBG without CRC and default COID. There is no OTP key blob generated. Optionally <Samples> specify the TRNG cycle samples for the TRNG Configure/Start (default=3072).
-ariaprovrndhuk256[=<Samples>]	Initialize the OTP with a random 256-bit HUK based on ARIA-DRBG with CRC and default COID. An OTP key blob with HUK is generated. Optionally <Samples> specify the TRNG cycle samples for the TRNG Configure/Start (default=3072).
-sleep	Set *EIP-130* in Sleep mode.
-stopsleep	Resume *EIP-130* from Sleep mode.
-hibernation	Set *EIP-130* in Hibernation mode.
-stophibernation	Resume *EIP-130* from hibernation mode. Note that this option can only be used in the combination '-hibernation -stophibernation'.
-hwmodulestatus	Get module status from hardware.
-hwoptions	Get hardware options.
-hwversion	Get hardware version information.
-selftest	Run self-test.
-fips	Run Enter-FIPS-Mode command.
-system	System information.
-reset	Reset *EIP-130*.

-regread=<Register>
Read an *EIP-130* internal register, where <Register> must be a hexadecimal representation of an address (e.g. 0xF878).

-regwrite=<Register>,<Value>
Write the specified <Value> to an EIP-130 internal register, where <Register> and <Value> must be a hexadecimal representation of an address or value (e.g. 0xF878).

-trng[=<Samples>]
Configure/Start the TRNG and enables an AES-DRBG. Optionally <Samples> specify the TRNG cycle samples (default=3072).

-trngrawdata[=<Blocks>]
Read raw data from TRNG/NRBG.
Optionally <Blocks> specify the number of data blocks (default=40), which is <Blocks>*256 bytes.

-ariadrbg[=<Samples>]
Configure/Start the TRNG and enables an ARIA-DRBG. Optionally <Samples> specify the TRNG cycle samples (default=3072).

-drbgforceseed Initialize the DRBG with a seed. Note that this command disables FROs use and reseeding.
Used Seed=0x635334C4CBDEC46E34B32096F0371198
DD4B395B443B24208BC935E6338A332B
ED88B88CDAA0CD174A5B0932A22E6E18

-zerootp Zeroize (destroy) OTP.

-sfwbotp Write SFWBPKD, SFWBCR and SFWBVER to OTP. Note that the OTP must be initialized with HUK and COID, and the EIP-130 firmware is reset to re-initialize the Static Asset references.

-firmwarecheck=filename
Run a firmware check on the specified firmware image file.

-updaterollbackid= RollbackId
Update the RollbackId (SFWBVER).

Note: The presence of some above-mentioned arguments depends on the HW/FW configuration with which the Demo Application is built.

Typical Demo Application use example

```
$ ./da_val_specials_up -emmcauthkey -trng -emmccheck
…
$ ./da_val_specials_up -ccounterotp -ccounterinc
…
$ ./da_val_specials_up -sleep -stopsleep
…
$ ./da_val_specials_up -zerootp
…
$ ./da_val_specials_up -provrndhuk256=5062
…
$ ./da_val_specials_up -milenageotp -trng -milenageautn -milenageauts
…
$ ./da_val_specials_up -hwmodulestatus -hwoptions -hwversion
…
$ ./da_val_specials_up -sfwbotp -firmwarecheck=./firmware_eip130ram.sbi
…
$ ./da_val_specials_up -reset -system
…
```

2.2.3 DemoApp_SecureDebug

The Demo Application `DemoApp_SecureDebug` (`da_securedebug_k.ko`, `da_securedebug_up` and `da_securedebug_combined_u[p]`) is used to show how to use the Authenticated Unlock and Secure Debug VAL API functions. The Demo Application is only built for user mode and the command line arguments control the program behavior. The following command line arguments can be specified:

-help Display this information.

-initotp Initialize the OTP with a default COID and HUK. After successful initialization of the COID and HUK, the *EIP-130* firmware is reset to re-initialize the Static Asset references.

-trng[=<Samples>]
Configure/Start the TRNG (default=3072).

-key=<AssetNumber>
Specifies the Asset number of the AuthKey. The Asset number range is [1…36].

-bits=<1024|2048|3072>
Specifies the modulus bit size. The bits range is [1024, 2048, 3072].

-exit Exit without running Secure Debug functionality.

Notes:

- *Depending on the HW/FW configuration, the Secure Debug functionality is based on ECDSA-256, ECDSA-521 or RSASSA-PKCS#1v1.5. This also means that the `-key` and `-bits` arguments are only present when the RSA algorithm is used.*
- *The `AssetNumber` that can be provided depends on the HW/FW configuration, please check the applicable range with the -help argument.*

Typical Demo Application use example in user mode:

```
$ ./da_securedebug_up                          # Algorithm depends on HW/FW cfg
…
$ ./da_securedebug_up -key=34 -bits=1024       # RSASSA-PKCS#1v1.5
…
$ ./da_securedebug_up -key=35 -bits=2048       # RSASSA-PKCS#1v1.5
…
$ ./da_securedebug_up -key=36 -bits=3072       # RSASSA-PKCS#1v1.5
…
```

2.2.4 DemoApp_OTPProgram

The Demo Application `DemoApp_OTPProgram` (`da_otpprogram_up` and `da_otpprogram_ combined_u[p]`) is used to show how to program the OTP using the VAL API functions. The Demo Application is only built for user mode and the command line arguments control the program behavior. The following command line arguments can be specified:

-help Display this information.

-verbose Enable logging.

blob <file> Specifies the file name of the key blob data file.

asset <number> Specifies the asset number of the asset [0…126] or [0…30].

policy <value> Specifies the policy (number) of the asset, see remark below.

label <data> Specifies the Associated Data that must be used.

crc Specifies that the asset must be protected with an CRC.

Depending on the FW configuration setting EIP130_FW_ASSETPOLICY_V2, the policy argument is defined either as (when EIP130_FW_ASSETPOLICY_V2 is defined):

policy <value> Specifies the Asset policy. (See *Firmware Reference Manual* [4] for details)
When <value> starts with:
'0x' a hexadecimal value is assumed
'0' an octal value is assumed
'[1..9]' a decimal value is assumed

Or (when EIP130_FW_ASSETPOLICY_V2 is not defined),

policy <number> Specifies the policy number of the asset:
1 - TRUSTED-KDK-DERIVE (HUK)
2 - AES key unwrap key (SB)
3 - TRUSTED-KEY-DERIVE (SB)
4 - Public Data (SB) for Public key or SHA256 Public key digest
6 - Auth Key for eMMC (AK)
7 - Private Data for Milenage K and OPc
8 - AES Unwrap key, (Secure Debug)
24 - AES key unwrap key (FWSB)
25 - TRUSTED-KEY-DERIVE (FWSB)
26 - SHA256 Public key digest (FWSB)
28 - JTAG Secure Debug Public Key (Secure Debug)

Typical Demo Application use example:

```
$ # EIP130_FW_ASSETPOLICY_V2 defined
$ ./da_otpprogram_up blob ./otpHUKblobPolicyValue.bin asset 97 \
policy 0x00050CC3 label "SomeAssociatedDataForProvisioningWithKeyBlob" crc \
–verbose
DA_OTPPRG: OTP Write information
Asset number : 97
Asset policy : 0x50CC3
AD Label     : SomeAssociatedDataForProvisioningWithKeyBlob
CRC          : Yes
OTP BlobData 00000000: 7E 5D 36 4E 25 7B 27 E4 0B 41 05 E9 5E C0 DE 99
OTP BlobData 00000016: A0 EA 96 0B 72 EB 95 37 60 34 44 A9 FE 62 DB 49
OTP BlobData 00000032: 89 63 38 2E 24 78 E7 17 81 E8 63 66 F5 E0 67 D8
DA_OTPPRG: PASSED: OTP data written.
NOTE: Hardware must be 'reset' to enable Asset for use.
$
```

```
$ # EIP130_FW_ASSETPOLICY_V2 not defined
$ ./da_otpprogram_up blob ./otpHUKblobPolicyNumber.bin asset 1 policy 1 \
label "SomeAssociatedDataForProvisioningWithKeyBlob" crc –verbose
DA_OTPPRG: OTP Write information
Asset number : 1
Asset policy : 1
AD Label     : SomeAssociatedDataForProvisioningWithKeyBlob
CRC          : Yes
OTP BlobData 00000000: 18 6E 2E A2 32 09 5B 4A 17 CD A0 DA 8C B8 88 ED
OTP BlobData 00000016: 2B 33 8A 33 E6 35 C9 8B 20 24 3B 44 5B 39 4B DD
OTP BlobData 00000032: 98 11 37 F0 96 20 B3 34 6E C4 DE CB C4 34 53 63
DA_OTPPRG: PASSED: OTP data written.
NOTE: Hardware must be 'reset' to enable Asset for use.
$
```

2.2.5 DemoApp_OTPKeyBlob

The Demo Application `DemoApp_OTPKeyBlob` (`da_otpkeyblob_up` and `da_otpkeyblob_ combined_u[p]`) is used to show how an OTP key blob can be generated using VAL API functions. The Demo Application is only built for user mode and the command line arguments control the program behavior. The following command line arguments can be specified:

-help	Display this information.
-verbose	Enable process logging.
key <file>	Specifies the file name of the binary file in which the AES-SIV key is given.
label <data>	Specifies the Associated Data that must be used.
policy <value>	Specifies the policy (number) of the asset, see remark below.
data <file>	Specifies the file name of the binary file in which the Asset data is given.
output <file>	Specifies the file name of the file to which the key blob data must be written.

Depending on the FW configuration setting EIP130_FW_ASSETPOLICY_V2, the policy argument is defined either as (when EIP130_FW_ASSETPOLICY_V2 is defined):

policy <value> Specifies the Asset policy. (See *Firmware Reference Manual* [4] for details)
When <value> starts with:
- '0x' a hexadecimal value is assumed
- '0' an octal value is assumed
- '[1..9]' a decimal value is assumed

Or (when EIP130_FW_ASSETPOLICY_V2 is not defined),

policy <number> Specifies the policy number of the asset:
- 1 - TRUSTED-KDK-DERIVE (HUK)
- 2 - AES key unwrap key (SB)
- 3 - TRUSTED-KEY-DERIVE (SB)
- 4 - Public Data (SB) for Public key or SHA256 Public key digest
- 6 - Auth Key for eMMC (AK)
- 7 - Private Data for Milenage K and OPc
- 8 - AES Unwrap key, (Secure Debug)
- 24 - AES key unwrap key (FWSB)
- 25 - TRUSTED-KEY-DERIVE (FWSB)
- 26 - SHA256 Public key digest (FWSB)
- 28 - JTAG Secure Debug Public Key (Secure Debug)

Typical Demo Application use example:

```
$
$ # EIP130_FW_ASSETPOLICY_V2 defined
$ ./da_otpkeyblob_up key ./ProvKey.bin policy 0x00050CC3 data ./HUK.bin \
label "SomeAssociatedDataForProvisioningWithKeyBlob" \
output ./otpHUKblobPolicyValue.bin
$
$ # EIP130_FW_ASSETPOLICY_V2 not defined
$ ./da_otpkeyblob_up key ./ProvKey.bin policy 1 data ./HUK.bin \
label "SomeAssociatedDataForProvisioningWithKeyBlob" \
output ./otpHUKblobPolicyNumber.bin
$
```

For reference the hexadecimal dumps of the involved binary files:

```
$ hd ./ProvKey.bin
00000000  18 e2 d1 78 1f 0c 1f 51  cc 3d 05 10 e6 f9 43 4b  |...x...Q.=....CK|
00000010  9e 0f 93 15 d7 44 6d 59  ac f3 00 73 f9 e7 35 e7  |.....DmY...s..5.|
00000020  fd fc 3a a0 60 48 a1 68  7c 48 77 17 be 3b fb ff  |..:.`H.h|Hw..;..|
00000030  18 30 10 e2 a7 2c 91 02  10 3c d6 6c 64 76 80 66  |.0...,...<.ldv.f|
$ hd ./HUK.bin
00000000  50 50 0a 0a ff aa 55 00  00 00 00 00 00 00 00 42  |PP....U........B|
00000010  50 50 0a 0a ff aa 55 00  00 00 00 00 00 00 00 42  |PP....U........B|
$ hd ./otpHUKblobPolicyValue.bin
00000000  7e 5d 36 4e 25 7b 27 e4  0b 41 05 e9 5e c0 de 99  |~]6N%{'..A..^...|
00000010  a0 ea 96 0b 72 eb 95 37  60 34 44 a9 fe 62 db 49  |....r..7`4D..b.I|
00000020  89 63 38 2e 24 78 e7 17  81 e8 63 66 f5 e0 67 d8  |.c8.$x....cf..g.|
$ hd ./otpHUKblobPolicyNumber.bin
00000000  18 6e 2e a2 32 09 5b 4a  17 cd a0 da 8c b8 88 ed  |.n..2.[J........|
00000010  2b 33 8a 33 e6 35 c9 8b  20 24 3b 44 5b 39 4b dd  |+3.3.5.. $;D[9K.|
00000020  98 11 37 f0 96 20 b3 34  6e c4 de cb c4 34 53 63  |..7.. .4n....4Sc|
$
```

3 Test Types

This chapter describes various test types implemented in the Test Tools for the driver verification as well as helper library verification.

The test types described in this chapter are used to verify the various functionality of the driver and the helper libraries. The driver is used to access and request services of the *EIP-130*. All tests are carried out using the Test Tools described in the previous chapter. The driver Test Tools execute tests that verify not only functional (using the *EIP-130* hardware in various modes) but also non-functional (performance and stability) aspects of the driver.

The Test Tool provides the basic mechanisms for testing the driver with pre-defined vectors.

3.1 Known Answer Test

The purpose of the Known Answer Test (KAT) is to verify the correctness of various operations and protocols supported by the *EIP-130*. The test compares the output of the operations performed by the *EIP-130* with provided known and correct values.

In principle, all tests that are performed for the *EIP-130* are of type KAT. If the test succeeds, the message “<test name> PASSED” is displayed otherwise “<test name> FAILED”.

3.2 Multiple Application Test

The multiple application test runs multiple instantiations of the test tool and demo application in parallel. The test tool and demo application perform known answer tests. The multiple application test passes when all test tool and demo application tests have passed.

4 Test Configuration Parameters

4.1 Driver Configuration Parameters

The driver static configuration defines how the driver accesses the *EIP-130*. This section describes the provided configuration parameters

4.1.1 Polling or Interrupt

This test configuration specifies whether the Test Tool should work synchronously or asynchronously to the device. In synchronous mode, referred to as polling, the Test Tool submits a request token and then waits for the device to produce the result token, which defines that the request token has been processed. While in asynchronous mode, the Test Tool submits a request token when the device mailbox is available for use and the device notifies to the Test Tool when the result token is available. In this mode of operation, interrupts are used.

4.1.2 Bounce Buffers

This test configuration is used to enable/disable the use of bounce buffers in the driver. If the configuration flag is set, the Test Tool allocates buffers on its own and registers the allocated buffers with the driver. The driver thereby allocates bounce buffers and copies the buffers received from the Test Tool to the new allocated buffers. The Test Tool calls `malloc` in user-space deployment and `kmalloc` in kernel-space deployment to allocate DMA-safe buffers and in this case, the driver does not allocate bounce buffers.

4.1.3 Performance

This test configuration is used to enable/disable strict argument checking in the driver. If the configuration flag is set, the strict argument checking is enabled.

4.1.4 Endianness Conversion

The *EIP-130* and the host processor may differ in their endianness. The *EIP-130* does not support the endianness conversion for its slave or master interface. The endianness conversion can be performed by the driver using the host processor instructions. For example, for the *EIP-130* to operate correctly it must receive the request token according to the specification in the *EIP-130* mailbox and data must have the correct endianness. For data used as byte stream, no endianness conversion is required.

4.1.5 Use TokenID to check DMA ready

This configuration setting is used to enable/disable the output DMA buffer check for the token ID value to ensure that the DMA operation has been completed before using the DMA data. It is a system-dependent setting when enabled, the driver will allocate a slightly bigger buffer for *EIP-130* output that is returned via the DMA.

Note: This configuration setting is not labeled as test configuration parameter. Make sure that this configuration setting is enabled when data corruption is detected. Thus, in doubt enable it.

4.2 Deployment Configurations

The driver may be used in two different deployments. The Test Tool and a Demo Application (`DemoApp_VAL`) can be used in any of these deployments:

- The driver and the Test Tool/Demo Application both run in the kernel space.
- The driver runs in the kernel space and the Test Tool/Demo Application in the user space.
- The driver and the Test Tool/Demo Application both run in the user space. The kernel support driver provides the user-space driver with the required Linux kernel services.

5 Supported tests and configurations

The *EIP-130* Verification Driver supports functionality of the *EIP-130*. The *EIP-130* driver can be tested with *EIP-130* hardware and firmware that is available in a FPGA, or a Verilog simulator with the *EIP-130* RTL, or a C-Model that emulates the *EIP-130* hardware.

Table 1 Supported Tests

Test	Supported Indication
Known Answer Test	Supported
Multiple Application Test	Supported

The driver must be tested in the following configurations to get 100% public API coverage and 80% code coverage. Note that when running only the regression, the coverage will not reach these goals due to the functionality that requires special OTP requirements or performs the OTP zeroization.

Table 2 Supported Configurations

Driver Configuration[1]	Polling / Interrupts	Bounce Buffers	Performance
C0	Polling	Off	Yes
C1	Interrupts	Off	Yes
C2	Polling	Off	No
C3	Polling	On	No
C4	Interrupts	On	Yes
C5	Interrupts	On	No

[1] Endianness Conversion and TokenID use to check DMA ready applies to all Driver Configurations when enabled.

6 Test Suite, Test Cases and Tests

A test suite is a combination of test cases, and a test case is a combination of tests, configuration parameters and one or more operations for execution by the driver. It represents a use case for the driver and is a unit for the system-level testing.

Currently there is one test suite “VAL_Tests”, which is setup (build up) automatically in VAL Test Tool (see section 2.1). This test suite is used to verify the VAL API and driver. This test suite holds the test cases and tests that are listed in this section related to the provided services.

Note: When the text is in red, the service or functionality is not yet supported, or the test case/test is not yet available.

6.1 No Operation services

This functionality is tested with the test case "NOP_Tests". The test case contains the following tests:

- test_nop
  The test validates the NOP operation.
- test_nop_unaligned_buffer
  The test validates the NOP operation, but the provided source and destination data references are located on an unaligned address. Note that this test only executed in user space.
- test_nop_arguments
  The test validates the error behavior of the function arguments. Note that this test is only executed when the strict argument checking is enabled.

6.2 Symmetric Crypto related services

The symmetric crypto functionality will be tested with and without Assets.

6.2.1 Encryption

Note: Certain algorithm requests can fail (be blocked) due to the mode of operation: non-FIPS or FIPS.

6.2.1.1 Cipher

The symmetric crypto cipher functionality is tested with the test case "SymCrypto_Cipher_Tests". The following algorithms are tested with reference vectors:

- AES
  - AES-ECB (128, 192, 256)
  - AES-CBC (128, 192, 256)
  - AEC-CTR (128, 192, 256)
  - AES-ICM (128, 192, 256)
  - XTS-AES (128, 256)
  - AES-*f*8 (128)
- DES (only allowed in non-FIPS mode)
- 3DES (only allowed in non-FIPS mode)
- ChaCha20 (Encrypt only)
- SM4
  - SM4-ECB
  - SM4-CBC
  - SM4-CTR

- ARIA
  - ARIA-ECB (128, 192, 256)
  - ARIA-CBC (128, 192, 256)
  - ARIA-CTR (128, 192, 256)
  - ARIA-ICM (128, 192, 256)

The test case contains the following tests for the AES related functionality:

- test_sym_crypto_aes
  The test validates the AES-ECB, AES-CBC, AES-CTR and AES-ICM operation with reference vectors. The result is verified. The test is performed 6 times where each time a certain attention point is tested. The attention points are:
  - Encrypt versus Decrypt
  - In-place (one buffer for In and Out) versus separate buffers
  - One single operation versus multipart operations
  - Key from token or Asset (when applicable)
- test_sym_crypto_aes_unaligned
  The test validates the AES-ECB, AES-CBC, AES-CTR and AES-ICM de/encrypt operations with reference vectors where the buffers are provided through an unaligned address. The result is verified. Note that this test only executed in user space.
- test_sym_crypto_xts_aes
  The test validates the XTS-AES operation with reference vectors. The result is verified. The test is performed 6 times where each time a certain attention point is tested. The attention points are listed under test_sym_crypto_aes.
- test_sym_crypto_aesf8
  The test validates the AES-*f*8 operation with reference vectors. The result is verified. The test is performed 4 times where each time a certain attention point is tested. The attention points are listed under test_sym_crypto_aes except multiple operations is not applicable for AES-*f*8.

The test case contains the following tests for the DES and Triple-DES related functionality:

- test_sym_crypto_des_3des
  The test validates the DES-ECB, DES-CBC, 3DES-ECB and 3DES-CBC operations with reference vectors. The result is verified. The test is performed 6 times where each time a certain attention point is tested. The attention points are:
  - Encrypt versus Decrypt
  - In-place (one buffer for In and Out) versus separate buffers
  - One single operation versus multipart operations
  - Key from token or Asset (when applicable)

The test case contains the following tests for the ChaCha20 related functionality:

- test_sym_crypto_chacha20
  The test validates the ChaCha20-Encrypt operations with reference vectors. The result is verified. The test is performed 6 times where each time a certain attention point is tested. The attention points are:
  - Encrypt versus Decrypt
  - In-place (one buffer for In and Out) versus separate buffers
  - One single operation versus multipart operations
  - Key from token or Asset
- test_sym_crypto_chacha20_keygen
  The test validates the ChaCha20 key generation operation in encrypt and decrypt direction with a reference vector. The result is verified.

The test case contains the following tests for all ciphers:

- test_sym_crypto_invalid_algorithm
  The test validates the invalid algorithm error situations.
- test_sym_crypto_buffer_too_small
  The test validates the buffer too small error situations.
- test_sym_crypto_common
  The test validates the error behavior of the arguments of all related functions. Note that this test is only executed when the strict argument checking is enabled.

The test case contains the following tests for the SM4 related functionality:

- test_sym_crypto_sm4
  The test validates the SM4-ECB, SM4-CBC and SM4-CTR operation with reference vectors. The result is verified. The test is performed 6 times where each time a certain attention point is tested. The attention points are listed under test_sym_crypto_aes.

The test case contains the following tests for the ARIA related functionality:

- test_sym_crypto_aria
  The test validates the ARIA-ECB, ARIA-CBC, ARIA-CTR and ARIA-ICM operation with reference vectors. The result is verified. The test is performed 6 times where each time a certain attention point is tested. The attention points are listed under test_sym_crypto_aes.

6.2.1.2 Cipher with Authenticated Encryption

The symmetric crypto cipher with Authenticated Encryption functionality is tested with the test case "SymCrypto_CipherAuth_Tests". The following algorithms are tested with reference vectors:

- AES
  - AES-CCM (128, 192, 256)
  - AES-GCM (128, 192, 256)
- ChaCha20
  - AEAD-ChaCha20-Poly1305
- ARIA
  - ARIA-CCM (128, 192, 256)
  - ARIA-GCM (128, 192, 256)
- SM4
  - SM4-GCM (128)

The test case contains the following tests for the AES related functionality:

- test_authenticated_crypto_aes_ccm
  The test validates the AES-CCM operation with reference vectors. The result is verified. The test is performed 6 times where each time a certain attention point is tested. The attention points are:
  - Encrypt versus Decrypt
  - In-place (one buffer for In and Out) versus separate buffers
  - Key from token or Asset (when applicable)
- test_authenticated_crypto_aes_gcm_auto
  The test validates the AES-GCM operation (autonomous) with reference vectors. The result is verified. The test is performed 6 times where each time a certain attention point is tested. The attention points are listed under test_authenticated_crypto_aes_ccm.
- test_authenticated_crypto_aes_gcm_H_pre_Y0_calc (N/A for *EIP-130* HW3.x)
  The test validates the AES-GCM operation (with H pre-calculated and Y0-encrypted calculated) with reference vectors. The result is verified. The test is performed 6 times where each time a certain attention point is tested. The attention points are listed under test_authenticated_crypto_aes_ccm.

- test_authenticated_crypto_aes_gcm_H_pre_Y0_0 (N/A for *EIP-130* HW3.x)
  The test validates the AES-GCM operation (with H pre-calculated and Y0-encrypted forced to 0) with reference vectors. The result is verified. The test is performed 6 times where each time a certain attention point is tested. The attention points are listed under test_authenticated_crypto_aes_ccm.
- test_authenticated_crypto_aes_gcm_ghash (N/A for *EIP-130* HW3.x)
  The test validates the AES-GCM operation (GHASH) with reference vectors. The result is verified. The test is performed 2 times, one time for Encrypt and Decrypt.

The test case contains the following tests for the ChaCha20 related functionality:

- test_authenticated_crypto_chacha20_aead
  The test validates the AEAD operation with a reference vector. The result is verified. The test is performed 6 times where each time a certain attention point is tested. The attention points are:
  - Encrypt versus Decrypt
  - In-place (one buffer for In and Out) versus separate buffers
  - Key from token or Asset

The test case contains the following tests for the ARIA related functionality:

- test_authenticated_crypto_aria_ccm
  The test validates the ARIA-CCM operation with reference vectors. The result is verified. The test is performed 6 times where each time a certain attention point is tested. The attention points are listed under test_authenticated_crypto_aes_ccm.
- test_authenticated_crypto_aria_gcm_auto
  The test validates the ARIA-GCM operation (autonomous) with reference vectors. The result is verified. The test is performed 6 times where each time a certain attention point is tested. The attention points are listed under test_authenticated_crypto_aes_ccm.
- test_authenticated_crypto_aria_gcm_H_pre_Y0_calc (N/A for *EIP-130* HW3.x)
  The test validates the ARIA-GCM operation (with H pre-calculated and Y0-encrypted calculated) with reference vectors. The result is verified. The test is performed 6 times where each time a certain attention point is tested. The attention points are listed under test_authenticated_crypto_aes_ccm.
- test_authenticated_crypto_aria_gcm_H_pre_Y0_0 (N/A for *EIP-130* HW3.x)
  The test validates the ARIA-GCM operation (with H pre-calculated and Y0-encrypted forced to 0) with reference vectors. The result is verified. The test is performed 6 times where each time a certain attention point is tested. The attention points are listed under test_authenticated_crypto_aes_ccm.
- test_authenticated_crypto_aria_gcm_ghash (N/A for *EIP-130* HW3.x)
  The test validates the ARIA-GCM operation (GHASH) with reference vectors. The result is verified. The test is performed 2 times, one time for Encrypt and Decrypt.

The test case contains the following tests for the SM4 related functionality:

- test_authenticated_crypto_sm4_gcm
  The test validates the ARIA-GCM operation (autonomous) with reference vectors. The result is verified. The test is performed 6 times where each time a certain attention point is tested. The attention points are:
  - Encrypt versus Decrypt
  - In-place (one buffer for In and Out) versus separate buffers
  - Key from token or Asset (when applicable)

6.2.2 Hash

The Hash functionality is tested with the test case "SymCrypto_Hash_and_HMac_Tests". The following algorithms are tested with reference vectors:

- SHA-1
- SHA-224
- SHA-256
- SHA-384
- SHA-512
- SHA-3-224
- SHA-3-256
- SHA-3-384
- SHA-3-512
- SM3

The test case contains the following tests for the Hash functionality:

- test_hash
  The test validates the Hash with reference vectors in one single operation. The Digest is verified.
- test_hash_multipart
  The test validates the Hash with reference vectors in multiple operations. The Digest is verified.
- test_hash_unaligned_buffer
  The test validates the Hash with reference vectors in one operation, but the message is provided through an unaligned address. The Digest is verified. Note that this test only executed in user space.
- test_hash_invalid_algorithm
  The test validates the invalid algorithm error situations.
- test_hash_buffer_too_small
  The test validates the buffer too small error situations.
- test_hash_arguments
  The test validates the error behavior of the arguments of all related functions. Note that this test is only executed when the strict argument checking is enabled.

6.2.3 MAC

6.2.3.1 Hash

The Hash based MAC functionality is tested with the test case "SymCrypto_Hash_and_HMac_Tests". The following algorithms are tested with reference vectors:

- HMAC-SHA-1
- HMAC-SHA-224
- HMAC-SHA-256
- HMAC-SHA-384
- HMAC-SHA-512
- HMAC-SHA-3-224
- HMAC-SHA-3-256
- HMAC-SHA-3-384
- HMAC-SHA-3-512
- HMAC-SM3

The Hash based MAC (HMAC) test case contains the following tests:

- test_hmac_generate
  The test validates the MAC generate operation with reference vectors in one single operation, where the key is provided via the token. The generated MAC value is verified.
- test_hmac_verify
  The test validates the MAC verify operation with reference vectors in one single operation, where the key and final MAC value is provided via the token.
- test_hmac_multipart_generate
  The test validates the MAC generate operation with reference vectors in multiple operations, where the key is provided via the token. The generated MAC value is verified.
- test_hmac_multipart_verify
  The test validates the MAC verify operation with reference vectors in multiple operations, where the key and final MAC value is provided via the token.
- test_hmac_generate_key_asset
  The test validates the MAC generate operation with reference vectors in one single operation, where the key comes from an Asset. The generated MAC value is verified.
- test_hmac_verify_key_asset
  The test validates the MAC verify operation with reference vectors in one single operation, where the key comes from an Asset and the final MAC value from the token.
- test_hmac_verify_mac_asset
  The test validates the MAC verify operation with reference vectors in one single operation, where the final MAC value comes from an Asset and the key from the token.
- test_hmac_verify_key_mac_asset
  The test validates the MAC verify operation with reference vectors in one single operation, where the key and final MAC value comes from Assets.
- test_hmac_unaligned_buffer
  The test validates the MAC operation with reference vectors in one operation, but the message is provided through an unaligned address. Note that this test only executed in user space.
- test_hmac_invalid_algorithm
  The test validates the invalid algorithm error situations.
- test_hmac_buffer_too_small
  The test validates the buffer too small error situations.
- test_hmac_arguments
  The test validates the error behavior of the arguments of all related functions. Note that this test is only executed when the strict argument checking is enabled.

6.2.3.2 Cipher

Note: Certain algorithm requests can fail (be blocked) due to the mode of operation: non-FIPS or FIPS.

The Cipher based MAC functionality is tested with the test case "SymCrypto_CipherMac_Tests". The following algorithms are tested with reference vectors:

- AES-CMAC
- AES-CBC-MAC
- Poly1305
- ARIA-CMAC
- ARIA-CBC-MAC

The Cipher based MAC test case contains the following tests:

- test_aes_cmac
  The test validates the AES-CMAC operation with reference vectors. The result is verified. The test is performed 16 times where each time a certain attention point is tested. The attention points are:
  - Generate versus Verify
  - One single operation versus multipart operations
  - Key from token or Asset
  - Final MAC from token or Asset
- test_aes_cbcmac
  The test validates the AES-CBC-MAC operation with reference vectors. The result is verified. The test is performed 16 times where each time a certain attention point is tested. The attention points are listed under test_aes_cmac.
- test_poly1305
  The test validates the Poly1305 operation with reference vectors. The result is verified. The test is performed 16 times where each time a certain attention point is tested. The attention points are listed under test_aes_cmac.
- test_aria_cmac
  The test validates the ARIA-CMAC operation with reference vectors. The result is verified. The test is performed 16 times where each time a certain attention point is tested. The attention points are listed under test_aes_cmac.
- test_aria_cbcmac
  The test validates the ARIA-CBC-MAC operation with reference vectors. The result is verified. The test is performed 16 times where each time a certain attention point is tested. The attention points are listed under test_aes_cmac.

6.2.4 Key Wrap

The Symmetric Key Wrap functionality is tested with the test case "SymCrypto_KeyWrapUnwrap_ Tests" and reference vectors. The test case contains the following tests:

- test_aes_wrap_unwrap
  The test validates the AES Key Wrap and Unwrap operation with reference vectors. The result is verified. The test is performed 2 times, one time for Wrap and Unwrap.
- test_sm4_wrap_unwrap
  The test validates the SM4 Key Wrap and Unwrap operation with reference vectors. The result is verified. The test is performed 2 times, one time for Wrap and Unwrap.

6.2.5 Encrypt Vector for PKI

The Encrypt Vector for PKI functionality is not tested as part the VAL Test Tool, but as part of a Demo Application.

6.3 Asymmetric Crypto related services

Note: Certain algorithm requests can fail (be blocked) due to the mode of operation: non-FIPS or FIPS.

6.3.1 With Asset use

The Public Key functionality with Assets use is tested with several test cases that cover all provided Asymmetric Cryptography functionality. The test cases and their tests are:

- AsymCrypto_DH_Vectors
  The test case contains the following tests for the Diffie-Hellman functionality:
  - test_dh_GenerateSharedSecret_L1024N160
    The test validates the shared secrets generation for the bit length pairs (1024, 160). The generated keys are checked with key check functionality.
  - test_dh_GenerateSharedSecret_L2048N224
    The test validates the shared secrets generation for the bit length pairs (2048, 224). The generated keys are checked with key check functionality.
  - test_dh_GenerateSharedSecret_L2048N256
    The test validates the shared secrets generation for the bit length pairs (2048, 256). The generated keys are checked with key check functionality.
  - test_dh_GenerateSharedSecret_L3072N256
    The test validates the shared secrets generation for the bit length pairs (3072, 256). The generated keys are checked with key check functionality.
- AsymCrypto_ECDSA_Vectors
  The test case contains the following tests for the Elliptic Curve Digital Signature Algorithm Sign and Verify functionality:
  - test_AsymEcdsaVectorsParamsCheck
    The test validates the curve parameters of the reference vectors. The curve parameters are loaded as an Asset and checked with the key check functionality.
  - test_AsymEcdsaGenKeyPair
    The test validates the ECDSA key pair generation with the reference vectors as input for the curve parameters and hash method. The result is verified key check functionality.
  - test_AsymEcdsaVectorsCheck
    The test validates the ECDSA verify with the reference vectors. The result is verified.
  - test_AsymEcdsaVectorsKeyPairCheck
    The test validates the ECDSA sign and verify operation with the reference vectors. The result is verified. Note that this test only executed when the TRNG is started.
  - test_AsymEcdsaVectorsKeyPairGenCheck
    The test validates the ECDSA sign and verify operation after the generation of a key pair. The result is verified. Note that this test only executed when the TRNG is started.
  - test_AsymEcdsaVectorsPublicKeyGenCheck
    The test validates the ECDSA sign and verify operation after the generation of a public key for the private key of the reference vector. The result is verified. Note that this test only executed when the TRNG is started.
  - test_AsymEcdsaInitKey
    The test validates the invalid algorithm error situations of the `val_AsymInitKey` function.
  - test_AsymEcdsaExplicitDigest
    The test validates the ECDSA sign and verify operation with the reference vectors, however the message is hashed first, and message digest is provided directly to the ECDSA sign and verify. The result is verified. Note that this test only executed when the TRNG is started.

  - test_AsymEcdsaArguments
    The test validates the error behavior of the arguments of all ECDSA related functions. Note that this test is only executed when the strict argument checking is enabled.

- AsymCrypto_ECDH_Vectors
  The test case contains the following tests for the Elliptic Curve Diffie-Hellman functionality:
  - test_ecdh_GenerateSharedSecret_192
    The test validates the shared secrets generation for the Elliptic Curve NIST P-192. The generated keys are checked with key check functionality.
  - test_ecdh_GenerateSharedSecret_224
    The test validates the shared secrets generation for the Elliptic Curve NIST P-224. The generated keys are checked with key check functionality.
  - test_ecdh_GenerateSharedSecret_256
    The test validates the shared secrets generation for the Elliptic Curve NIST P-256. The generated keys are checked with key check functionality.
  - test_ecdh_GenerateSharedSecret_384
    The test validates the shared secrets generation for the Elliptic Curve NIST P-384. The generated keys are checked with key check functionality.
  - test_ecdh_GenerateSharedSecret_521
    The test validates the shared secrets generation for the Elliptic Curve NIST P-521. The generated keys are checked with key check functionality.
  - test_ecdh_GenerateSharedSecret_PublicData
    The test validates whether the shared secret can be placed in a public data asset, based on the ST_ANY policy bit.
- AsymCrypto_ECC_ElGamal_Vectors
  The test case contains the following tests for the Elliptic Curve Cryptography El-Gamal Encryption and Decryption functionality:
  - test_AsymEccElgamalVectorsParamsCheck
    The test validates the curve parameters of the reference vectors. The curve parameters are loaded as an Asset and checked with the key check functionality.
  - test_AsymEccElgamalVectorsCheck
    The test validates the ECC El-Gamal Decryption with the reference vectors. The result is verified.
  - test_AsymEccElgamalVectorsKeyPairCheck
    The test validates the ECC El-Gamal Encryption and Decryption operation with the reference vectors. The result is verified. Note that this test only executed when the TRNG is started.
  - test_AsymEccElgamalVectorsKeyPairGenCheck
    The test validates the ECC El-Gamal Encryption and Decryption operation after the generation of a key pair. The result is verified. Note that this test only executed when the TRNG is started.
  - test_AsymEccElgamalVectorsPublicKeyGenCheck
    The test validates the ECC El-Gamal Encryption and Decryption operation after the generation of a public key for the private key of the reference vector. The result is verified. Note that this test only executed when the TRNG is started.
  - test_AsymEccElgamalInitKey
    The test validates the invalid algorithm error situations of the `val_AsymInitKey` function.
  - test_AsymEccElgamalArguments
    The test validates the error behavior of the arguments of all ECC El-Gamal related functions. Note that this test is only executed when the strict argument checking is enabled.

- AsymCrypto_Curve25519_Vectors
  The test case contains the test for the shared secrets generation using the Curve 25519:
  - test_curve25519
    The test validates the shared secrets generation for the Elliptic Curve 25519. The generated keys are checked with key check functionality.
- AsymCrypto_EdDSA_Vectors
  The test case contains the test for the Edwards Digital Signature Algorithm Sign and Verify functionality:
  - test_AsymEddsaVectorsCheck
    The test validates the EdDSA verify with the reference vectors. The test is performed 4 times with following attention points:
    - An EdDSA verify operation is performed with a correct signature, which must pass.
    - The EdDSA sign and verify combination is performed with the message, private and public key of the reference vectors.
    - A key pair is generated and an EdDSA sign and verify combination is performed with the message of the reference vectors.
    - The public key is generated and an EdDSA sign and verify combination is performed with the message and private key of the reference vectors.
  - test_AsymEddsaLongMessageCheck
    The test performs an EdDSA sign and verify combination with a 10k message based on a generated key pair.
- AsymCrypto_RSA_Vectors
  The test case contains the following tests for the RSA based functionality:
  - test_rsa_pkcs1v15_1024
    The test validates the RSA PKCS#1 v1.5 sign and verify operation with the reference vectors for a 1024-bit modulus. The result is verified. Note that this test only executed when the TRNG is started.
  - test_rsa_pkcs1v15_2048
    The test validates the RSA PKCS#1 v1.5 sign and verify operation with the reference vectors for a 2048-bit modulus. The result is verified. Note that this test only executed when the TRNG is started.
  - test_rsa_pkcs1v15_3072
    The test validates the RSA PKCS#1 v1.5 sign and verify operation with the reference vectors for a 3072-bit modulus. The result is verified. Note that this test only executed when the TRNG is started.
  - test_rsa_pss_1024
    The test validates the RSA-PSS sign and verify operation with the reference vectors for a 1024-bit modulus. The result is verified. Note that this test only executed when the TRNG is started.
  - test_rsa_pss_2048
    The test validates the RSA-PSS sign and verify operation with the reference vectors for a 2048-bit modulus. The result is verified. Note that this test only executed when the TRNG is started.
  - test_rsa_pss_3072
    The test validates the RSA-PSS sign and verify operation with the reference vectors for a 3072-bit modulus. The result is verified. Note that this test only executed when the TRNG is started.
  - test_rsa_oaep_1024
    The test validates the RSA-OAEP wrap and unwrap operation with the reference vectors for a 1024-bit modulus. The result is verified. Note that this test only executed when the TRNG is started.

  - test_rsa_oaep_2048
    The test validates the RSA-OAEP wrap and unwrap operation with the reference vectors for a 2048-bit modulus. The result is verified. Note that this test only executed when the TRNG is started.
  - test_rsa_oaep_3072
    The test validates the RSA-OAEP wrap and unwrap operation with the reference vectors for a 3072-bit modulus. The result is verified. Note that this test only executed when the TRNG is started.
  - test_rsa_pkcs1v15wrap_1024
    The test validates the RSA-PKCS#1v1.5 wrap and unwrap operation with the reference vectors for a 1024-bit modulus. The result is verified. Note that this test only executed when the TRNG is started.
  - test_rsa_pkcs1v15wrap_2048
    The test validates the RSA-PKCS#1v1.5 wrap and unwrap operation with the reference vectors for a 1024-bit modulus. The result is verified. Note that this test only executed when the TRNG is started.
  - test_rsa_pkcs1v15wrap_3072
    The test validates the RSA-PKCS#1v1.5 wrap and unwrap operation with the reference vectors for a 1024-bit modulus. The result is verified. Note that this test only executed when the TRNG is started.
  - test_rsa_pkcs1v15wrap_PublicData
    The test validates whether the unwrapped data can be placed in a Public Data Asset, based on the private key policy.
- AsymCrypto_ECIES
  - test_AsymECIES
    The test validates the ECIES wrap and unwrap operation with predefined P-256, P-384 and P-521 keys. The result is verified. Note that this test is only executed when the TRNG is started.
  - test_AsymEciesStrictArguments
    The test validates the error behavior of the arguments of all ECIES related functions. Note that this test is only executed when the strict argument checking is enabled.
- AsymCrypto_SM2_Vectors
  - test_AsymSm2dsaSignVerifyVectors
    The test validates the SM2DSA sign and verify operation with the reference vectors.
  - test_AsymSm2dsaSignVerifyLongMessage
    The test validates the SM2DSA sign and verify operation with the reference vectors, but with a long message as input.
  - test_AsymSm2kexGenerateSharedSecret
    The test validates the SM2 shared secrets generation. The generated keys are checked by loading the secret into a public data asset and compared to the expected result.
  - test_AsymSm2kexGenerateSharedSecret_PublicData
    The test validates whether a generated shared secret can be placed in a Public Data Asset, based on the private key policy.
  - test_AsymSm2kexFp192RandomKeys
    The test validates the SM2 key generation. Instead of using reference vectors to create a shared secret, random keypairs will be generated and used instead.
  - test_AsymSm2enc
    The test validates the SM2enc Wrap- Unwrap operations using random generated keypairs.
  - test_AsymSm2encArguments
    The test validates the error behavior of the arguments of all SM2enc related functions. Note that this test is only executed when the strict argument checking is enabled.

- test_AsymSm2dsaArguments
  The test validates the error behavior of the arguments of all SM2dsa related functions. Note that this test is only executed when the strict argument checking is enabled.
- test_AsymSm2kexArguments
  The test validates the error behavior of the arguments of all SM2kex related functions. Note that this test is only executed when the strict argument checking is enabled.

Note: For DH / ECDH /ECDSA / ECC El-Gamal / Curve25519 / EdDSA, the functionality to “generate public key”, “generate private and public key” and “check the private and /or public key” are tested as part of the tests for the specific method.

6.3.2 Without Asset use

The Public Key functionality without Asset(s) use is tested with the test case "AsymCrypto_Pk WithoutAssets" and reference vectors. The test case contains the following test:

- test_AsymPkModExpE
  The test validates the Encrypt Modular Exponentiation operation with reference vectors. The result is verified.
- test_AsymPkModExpD
  The test validates the Decrypt Modular Exponentiation operation with reference vectors. The result is verified.
- test_AsymPkModExpCRT
  The test validates the CRT Modular Exponentiation operation with reference vectors. The result is verified.
- test_AsymPkEcMontMul
  The test validates the Elliptic Curve Montgomery Multiplication operation with reference vectors. The result is verified.
- test_AsymPkEccAddA
  The test validates the Elliptic Curve Cryptography Addition operation with reference vectors. The result is verified.
- test_AsymPkEccMulA
  The test validates the Elliptic Curve Cryptography Multiplication operation with reference vectors. The result is verified.
- test_AsymPkDsaSign
  The test validates the Digital Signature Algorithm Sign operation with reference vectors. The result is verified.
- test_AsymPkDsaVerify
  The test validates the Digital Signature Algorithm Verify operation with reference vectors. The result is verified.
- test_AsymPkEcdsaSign
  The test validates the Elliptic Curve Digital Signature Algorithm Sign operation with reference vectors. The result is verified.
- test_AsymPkEcdsaVerify
  The test validates the Elliptic Curve Digital Signature Algorithm Verify operation with reference vectors. The result is verified.

Note that above-mentioned tests used the `val_AsymPkaClaim()`, `val_AsymPkaLoadVector()`, `val_AsymPkaExecuteOperation()` and `val_AsymPkaClaimRelease()` functions to perform the specified operations. Additionally, the functions `val_Claim()` and `val_ClaimRelease()` are tested as part of the `val_AsymPkaClaim[Release]()` functions.

6.4 TRNG (Random) related services

The test case “Random_Tests” holds all tests for the provided TRNG services as listed below:

- TRNG (NRBG/DRBG) Configuration
  This functionality is tested with the test ‘test_rand_config’.
- DRBG Reseed
  This functionality is tested as part of random number generation, test ‘test_rand’. Note this test is only performed when the calling Host (the CPU on which tests are executed) has a Secure connection with *EIP-130*.
- DRBG Get Random Number
  This service is tested with the tests ‘test_rand’ and ‘test_rand_verylong’. For the test ‘test_rand’, multiple number of random numbers are quested after an optional reseed operation. When a total of 128Kbytes random data is retrieved, the randomness is determined. The test ‘test_rand_verylong’ retrieves roughly 8Mbytes of random data.
- DRBG Post-Processing verification
  This functionality is tested with the following three tests:
  - test_TrngVerPP_CF_12bitRepeatPattern
    The operation is performed with a reference pattern and the result is verified.
  - test_TrngVerPP_CF_ExternalNoise
    The operation is performed with a reference input data and noise. The result is verified.
  - test_TrngVerPP_DRBG_KnownData
    The operation is performed with a reference vector and the result is verified.

  Note that these tests are only performed when the calling Host (the CPU on which tests are executed) has a Secure connection with *EIP-130*.
- NRBG Self-Test
  This functionality is tested with the following two tests that use a reference pattern:
  - test_TrngVerHW_128BytesPattern
  - test_TrngVerHW_8BytesPatternWithPropRep

  Note that these tests are only performed when the calling Host (the CPU on which tests are executed) has a Secure connection with *EIP-130*.

When strict argument checking is enabled, the arguments of all TRNG related function are validated with the test ‘test_rand_arguments’.

6.5 Asset Management related services

6.5.1 Asset related

The test case “AssetManagement_Tests” holds all tests for the provided services listed below:

- Asset Search
  This service is tested with two tests that cover all possible functionality. The tests are:
  - test_AssetSearch
    Tests the VAL API function “val_AssetSearch”.
  - test_AssetGetRootKey
    Tests the VAL API function “val_AssetGetRootKey”.
- Asset Create
  This service is tested with multiple tests that cover all possible functionality. In general, the test ‘test_AssetAllocFree’ is used to test it.

- Asset Load
  The tests for this service are performed per load method:
  - Derive
    This Asset Load method is tested with test ‘test_AssetLoadDerive’. The various possible derive methods are tested with this test.
  - Plaintext
    This Asset Load method is tested with test ‘test_AssetLoadPlaintext’. The test is executed twice the first time (0) the Asset is only load and the second time (1) the Asset is loaded and exported as a (AES-SIV) key blob.
  - Random
    This Asset Load method is tested with test ‘test_AssetLoadRandom’. The test is executed twice the first time (0) the Asset is only load and the second time (1) the Asset is loaded and exported as a (AES-SIV) key blob. Note that the test will return unsupported if the TRNG is not started.
  - Import
    This Asset Load method is tested with test ‘test_AssetLoadImport’. The test first creates a (AES-SIV) key blob that is used in the import operation.
  - Unwrap
    This Asset Load method is tested with test ‘test_AssetLoadSymUnwrap_aes’ and ‘test_AssetLoadSymUnwrap_sm4’. The test first creates a symmetric wrapped key blob that is used in the Symmetric Unwrap operation.
- Asset Delete
  This service is tested with multiple tests that cover all possible functionality. In general, the test ‘test_AssetAllocFree’ is used to test it.
- Asset Export via Co-Processor interface
  This service is tested with ‘test_AssetExportCPIF’. The test creates multiple assets and sends them to various co-processors. Note the test can result in undefined behavior on real hardware.

6.5.2 Secure Timer

The test case “SecureTimer_Tests” holds all tests for the Secure Timer services listed below:

- test_secure_timer
  This test validates all Secure Timer functions. It creates and starts a Second and 100µs timer. After a certain time, the elapsed time of the Second timer is read. At the end of the test the Second timer is stopped and the 100µs timer is released.
- test_secure_timer_restart
  This test creates a 100µs timer, restarts it after a certain time and again after a certain time the elapsed time is read and validated.

6.5.3 OTP

The OTP related functionality is not tested as part the VAL Test Tool, but as part of a Demo Application.

6.6 Authentication Unlock/Secure Debug related services

The Authentication Unlock and Secure Debug related functionality is not tested as part the VAL Test Tool, but as part of a Demo Application.

6.7 eMMC related services

The eMMC related functionality is not tested as part the VAL Test Tool, but as part of a Demo Application.

6.8 Special Functions related services

The functionality is mainly tested with the test case "SpecialFunctions_Tests".

6.8.1 Milenage

For Milenage the conformance check functionality is only tested as part the VAL Test Tool. The test case "SpecialFunctions_Tests" contains the test “test_SF_MilenageConformance”. The test tests all conformance test data that is specified in the 3GPP TS 35.208 (V13.0.0).

The standard use Milenage functionality is not tested as part the VAL Test Tool, but as part of a Demo Application.

6.9 Service-related services

This functionality is tested with the test case "Service_Tests". The test case contains the following tests:

- test_service_ReadWriteRegister
  The test validates the register read and write functionality.
- test_service_ZeroizeMailbox
  The test calls the `val_ServiceZeroizeMailbox()` function to zeroize the output mailbox.
- test_service_ClockSwitch
  The test manipulates the hash clock and validates that hash operation fails with a clock error when clock is disabled and passes when clock is enabled again.

Note: The OTP Zeroization functionality is not tested as part of the VAL Test Tool.

6.10 System related services

The following system related functionality is tested:

- System Information
  This functionality is tested as part of the test case "System_Tests" with following tests:
  - test_system_version
    The test retrieves the hardware and firmware version via the `val_SystemGetVersion` function.
  - test_system_state
    The test retrieves the system state information via the `val_SystemGetState` function.
- Reset
  This functionality is tested as part of the test case "System_Tests" with the test ‘test_system_reset’.
- Set Time
  This functionality is tested as part of the test case "System_Tests" with the test ‘test_system_set_time’. This test is used in combination with creation of assets that specify Asset Lifetime. Note that a secure Host with COID must perform the operation.
- Define Users
  This functionality is tested as part of the test case "FIPS_Tests" with the test ‘test_fips_system_define_users1’ in non-FIPS mode and when system reset is allowed in FIPS mode with ‘test_fips_system_define_users2’.
- Self-Test
  This functionality is tested as part of the test case "FIPS_Tests" when system reset is allowed with the test ‘test_fips_system_selftest’.
- Enter-FIPS-Mode
  This functionality is tested as part of the test case "FIPS_Tests" when system reset is allowed with the test ‘test_fips_system_enterfipsmode’.

Note: The Sleep and Hibernation functionality is not tested as part of the VAL Test Tool but can be tested with the VAL Specials Demo Application.

(End of Document)